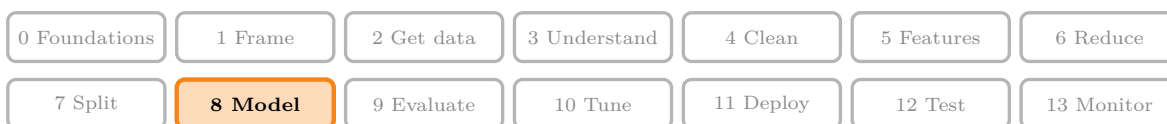


Introduction to Random Forest

Contents

1. Overview	2
2. Why random forests are so popular	3
3. Where the name comes from	3
4. How a random forest works	3
5. A random forest by hand	4
5.1 The data and the sampling functions	4
5.2 Row sampling	5
5.3 Column sampling and combined sampling	5
5.4 What a real random forest does differently	6
6. Summary	7
7. Key terms	7

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Overfitting ([Note 91](#)); Hyperparameter tuning ([Note 98](#)); Decision trees ([Note 100](#)); Feature importance ([Note 100](#)); Bagging ([Note 107](#)); OOB score ([Note 107](#)).
- **Leads to:** Balanced random forest ([Note 133](#)).
- **Compare with:** Bagging ([Note 107](#)).

1. Overview

Key point: A random forest trains many decision trees, each on a random part of the data, and combines their answers by majority vote (classification) or mean (regression).

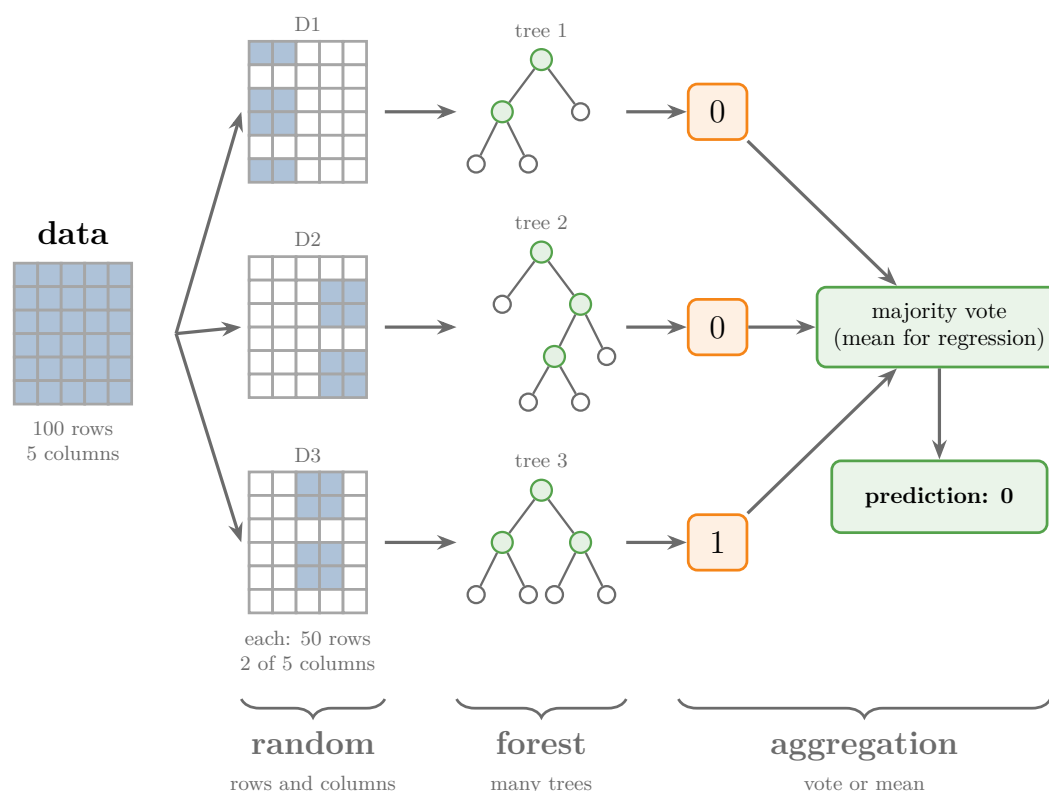


Figure 1: A random forest: each tree gets its own random rows and columns, grows differently, and votes

A **random forest** is bagging (the [bagging Note](#)) with decision trees (the [decision trees Note](#)) as the base models. Figure 1 shows the whole idea: random subsets of the data go to different trees, each tree predicts, and the forest takes the majority.

This Note covers why random forests are so widely used, where the name comes from, how one works, and a small forest of three trees built by hand. The next Notes explain why it works so well ([random forest and bias-variance](#)), how it differs from plain bagging ([bagging vs random forest](#)), its hyperparameters, tuning, the OOB score and feature importance.

The Notebook ([notebook.ipynb](#)) runs the forest built by hand.

2. Why random forests are so popular

Key point: Strong results on almost any problem, classification and regression alike, with little or no tuning.

Three things make the random forest one of the first algorithms to try in any project:

1. **It performs well almost everywhere.** Together with gradient boosting and XGBoost (in later Notes), it is one of the few algorithms whose results stay strong across very different problems.
2. **It handles both problem types.** `RandomForestClassifier` predicts classes and `RandomForestRegressor` predicts numbers.
3. **It needs little tuning.** With every setting left at its default, it often gives the best result of all the algorithms we try. This makes it especially friendly for beginners.

Extra: The random forest was published by the statistician Leo Breiman in 2001, in a paper titled *Random Forests*. Breiman had also introduced bagging in 1996; his forests built on Tin Kam Ho's 1995 idea of "random decision forests", trees trained on random subsets of the columns.

3. Where the name comes from

Key point: "Forest" because it is a collection of trees; "random" because each tree gets randomly sampled data.

- **Forest:** we train many decision trees together. A group of trees is a forest.
- **Random:** a random forest is a bagging technique, and bagging means **bootstrap aggregation**. Bootstrapping draws a random sample of the data for each tree (the [bagging Note](#), section 2.1). That random selection gives the "random".

So a random forest is a special case of bagging: bagging lets the base model be any algorithm, and when that algorithm is a decision tree, we call the result a random forest. There is one more, subtler difference, the subject of the [bagging vs random forest Note](#).

4. How a random forest works

Key point: Step 1, bootstrapping: give each tree a random subset of the rows, columns or both, and train it. Step 2, aggregation: send a new point to every tree and take the vote or the mean.

The two steps are those of bagging (the [bagging Note](#), sections 2 and 6), here with trees:

1. **Bootstrapping:** each tree is trained on its own random subset of the data, made by **row sampling** (random rows, all columns), **column sampling**, also called **feature sampling** (all rows, random columns), or **combined sampling** (both). These are the bagging, random subspaces and random patches types of the bagging Note, section 6.
2. **Aggregation:** a new query point goes to every tree, and the forest returns the majority vote (classification) or the mean (regression), as every ensemble does (the [ensemble learning Note](#), section 3).

Because every tree sees different data, every tree learns a different structure: one splits first on column 3, another on column 7, and they grow to different shapes (Figure 1, middle).

5. A random forest by hand

Key point: Three trees, each trained on its own random subset of a 100-row dataset, vote on one query point; with row, column or combined sampling, the majority gets it right even when one tree is wrong.

To see every step, we build a forest of three trees ourselves. This is a simplified version: a real random forest uses many more trees and samples the columns in a slightly different way (section 5.4).

5.1 The data and the sampling functions

Key point: 100 rows, 5 input columns, two classes; three small functions draw the subsets.

We make a classification dataset of 100 rows with `make_classification` (the [perceptron code Note](#)): 5 input columns, `col1` to `col5`, and a `target` column with classes 0 and 1.

Python: The three ways of sampling.

```
def sample_rows(df, share):
    n = int(share * len(df))
    return df.sample(n, replace=True)

def sample_features(df, share):
    # do not count the target column
    n = int(share * (df.shape[1] - 1))
    cols = list(rng.choice(df.columns[:-1], n,
                           replace=False))
    return df[cols + ["target"]]

def combined_sampling(df, row_share, col_share):
    rows = sample_rows(df, row_share)
    return sample_features(rows, col_share)
```

`df.sample(n, replace=True)` draws `n` rows with replacement (the [bagging Note](#), section 5.2). `rng.choice(columns, n, replace=False)` picks `n` different column names at

random; `rng` is NumPy's random generator, `np.random.default_rng(4)`, seeded so every run draws the same subsets. The target column is added back, because every tree needs it to learn.

Each tree is a fully grown `DecisionTreeClassifier`, trained on its subset's input columns. The query point is row 7 of the data, whose true class is **0**.

5.2 Row sampling

Key point: 20 rows per tree, with replacement: the votes are 1, 0, 0, so the forest says 0.

With `sample_rows(df, 0.2)`, each tree gets 20 rows (20% of 100), drawn with replacement:

Tree	Rows	Repeated rows	Depth	Vote
1	20	2	2	1
2	20	1	2	0
3	20	1	2	0

Tree 1 is wrong, but two trees say 0, so the majority, **0**, is right. Each tree saw different rows, so each drew its boundaries in different places.

5.3 Column sampling and combined sampling

Key point: 4 of 5 columns per tree: all three trees say 0. Half the rows and 2 of 5 columns: the votes are 0, 0, 1, so the forest still says 0.

With `sample_features(df, 0.8)`, each tree gets all 100 rows and 4 of the 5 columns (80% of 5). The three trees got different column sets: (col5, col4, col3, col2), (col5, col2, col1, col3) and (col1, col3, col2, col5). With all the rows, the trees grow deeper (depths 6, 3 and 3), and all three vote **0**.

With `combined_sampling(df, 0.5, 0.5)`, each tree gets 50 rows (10 to 14 of them repeats) and 2 of the 5 columns (half of 5 is 2.5, and `int` rounds down to 2). This is Figure 1:

Tree	Columns	Depth	Vote
1	col2, col1	4	0
2	col5, col4	5	0
3	col4, col3	6	1

This setting adds the most randomness: the trees differ in both rows and columns. The majority is again **0**, the right answer.

Python: Asking one tree about the query point.

```
inputs = d.columns[:-1]    # this tree's input columns
tree = DecisionTreeClassifier().fit(d[inputs], d["target"])
tree.predict(query[inputs])
```

A tree trained on 2 columns must be given exactly those 2 columns when it predicts. Selecting `query[inputs]` passes the same columns in the same order.

5.4 What a real random forest does differently

Key point: scikit-learn's `RandomForestClassifier` does all this in one line, with 100 trees by default, and samples the columns afresh at every split.

Python: A random forest in scikit-learn.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state=42)
rf.fit(df.iloc[:, :5], df["target"])
rf.predict(query.iloc[:, :5])    # array([0])
```

The forest of 100 trees also predicts **0**. Two things differ from our hand-built version:

- **More trees.** Three trees are only enough to show the idea; the default is 100.
- **Column sampling at every split.** Our functions picked the columns once per tree. A random forest picks a fresh random set of columns at every node of every tree, which adds even more variety. This is the main difference between bagging and a random forest, explained in the [bagging vs random forest Note](#).

6. Summary

Step	What happens	In our example
Bootstrapping	each tree gets a random subset: rows, columns or both	3 trees; 20 rows, or 4 columns, or 50 rows and 2 columns each
Training	each subset trains its own fully grown decision tree	trees of depth 2 to 6, all different
Aggregation	majority vote (classes) or mean (numbers)	votes 1, 0, 0 and 0, 0, 1: prediction 0

- A random forest is bagging with decision trees as the base models.
- “Forest”: many trees. “Random”: each tree gets randomly sampled data.
- Subsets can be made by row sampling, column sampling or both, with or without replacement.
- It works for classification and regression, and gives strong results with little tuning.

7. Key terms

Term	Meaning
Random forest	Bagging with decision trees as the base models; the trees vote (classification) or are averaged (regression)
Row sampling	Giving each base model a random subset of the rows
Column sampling (feature sampling)	Giving each base model a random subset of the columns
Combined sampling	Giving each base model random rows and random columns together
RandomForestClassifier	scikit-learn’s random forest for classification
RandomForestRegressor	scikit-learn’s random forest for regression